

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

[https://github.com/dv0retsky/fastapi-](https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI2_Endpoints%20and%20Parameters/FAPI2_Endpoints%20and%20Parameters.md)

[tutorial/blob/main/FAPI2_Endpoints%20and%20Parameters/FAPI2_Endpoints%20and%20Parameters.md](https://github.com/dv0retsky/fastapi-tutorial/blob/main/FAPI2_Endpoints%20and%20Parameters/FAPI2_Endpoints%20and%20Parameters.md)

Практическое занятие №2: Работа с конечными точками и параметрами

API (англ. **application programming interface** — **программный интерфейс приложения**) — это набор способов и правил, по которым различные программы общаются между собой и обмениваются данными. Все эти взаимодействия происходят с помощью функций, классов, методов, структур, а иногда констант одной программы, к которой обращаются другие. Это основной принцип работы **API**.

REST API — это архитектурный подход для создания веб-сервисов. Он прописывает ограничения устройства и функциональности **API**. По сути, **REST** — это набор правил и принципов. Всего их шесть:

- **Клиент-серверная архитектура.** При этом подходе приложение состоит из двух «ипостастей»: сервера и клиента.
- **Отсутствие состояния (statelessness).** Сервер не хранит данные о предыдущих взаимодействиях с клиентом — каждый запрос от клиента к серверу должен быть самодостаточным. При таком подходе нагрузка на сервер снижается в несколько раз — ему не нужно держать в памяти каждую сессию обмена данными с клиентом.
- **Кэшируемость (cacheability).** Благодаря кэшируемости клиент может сохранять ответы API для повторного использования без выполнения запроса. Это снижает задержку и нагрузку на сервер. Ответы REST API также должны указывать, могут ли они быть кэшированы клиентом.
- **Единообразие интерфейса (uniform interface).** REST API должны работать через единообразный интерфейс, который упрощает взаимодействие между клиентами и серверами. В **REST API** это стандартные методы **HTTP (GET, POST, PUT, DELETE)**, а также чёткие соглашения об именовании ресурсов.
- **Слои (layered system).** Архитектура **REST API** — это настоящий слоёный пирог, который состоит из серверов, прокси, шлюзов, балансировщиков нагрузки и массы других

вспомогательных систем. Поэтому клиенты никогда не знают точно, общаются они напрямую с сервером или с его «дублёром».

- **Код по требованию (code on demand)** — необязательный принцип. Сервер может отправлять код, который будет выполняться уже на стороне клиента, — например, это может быть код JS-скриптов и других интерактивных элементов интерфейса.

Протокол передачи гипертекста (Hypertext Transfer Protocol - HTTP) — это прикладной протокол для передачи гипертекстовых документов, таких как **HTML**. Он создан для связи между веб-браузерами и веб-серверами, хотя в принципе **HTTP** может использоваться и для других целей. Протокол следует классической клиент-серверной модели, когда клиент открывает соединение для создания запроса, а затем ждёт ответа. **HTTP** — это протокол без сохранения состояния, то есть сервер не сохраняет никаких данных (состояние) между двумя парами "запрос-ответ". Несмотря на то, что **HTTP** основан на **TCP/IP**, он также может использовать любой другой протокол транспортного уровня с гарантированной доставкой.

Так как **HTTP** — это клиент-серверный протокол, **HTTP** сессия состоит из трёх фаз:

1. Клиент устанавливает **TCP** соединения (или другое соединение, если не используется **TCP** транспорт).
2. Клиент отправляет запрос и ждёт ответа.
3. Сервер обрабатывает запрос и посылает ответ, в котором содержится код статуса и соответствующие данные.

HTTP-запросы

GET. С помощью него клиент запрашивает у сервера содержимое ресурса. Например, HTML-страницу. Кроме того, с помощью этого метода на сервер можно передать данные клиента. Для этого в адресе ресурса после символа ? необходимо добавить параметры и их значения, которые нужно передать на сервер.

HEAD. Это метод для получения заголовков ресурса. Часто его применяют для получения метаданных и проверки того, менялся ли ресурс с момента последнего посещения и существует ли он.

POST. С помощью этого метода клиент может передать данные в теле сообщения, например, в случаях, когда данные нельзя передать с помощью метода GET. Это может быть пост в соцсети или данные банковской карты, которую нежелательно оставить в истории поиска. Также **POST** является неидемпотентным методом: при его отправке результат может отличаться. Ещё одна особенность **POST** — ответы на него не будут кэшироваться.

OPTIONS. С помощью него можно запросить список методов, которые он или его ресурс поддерживает. Также **OPTIONS** можно использовать для того, чтобы «пропинговать» сервер — протестировать его работоспособность.

PUT. Этот метод создаёт новый ресурс или заменяет существующий данными, которые указаны в теле запроса.

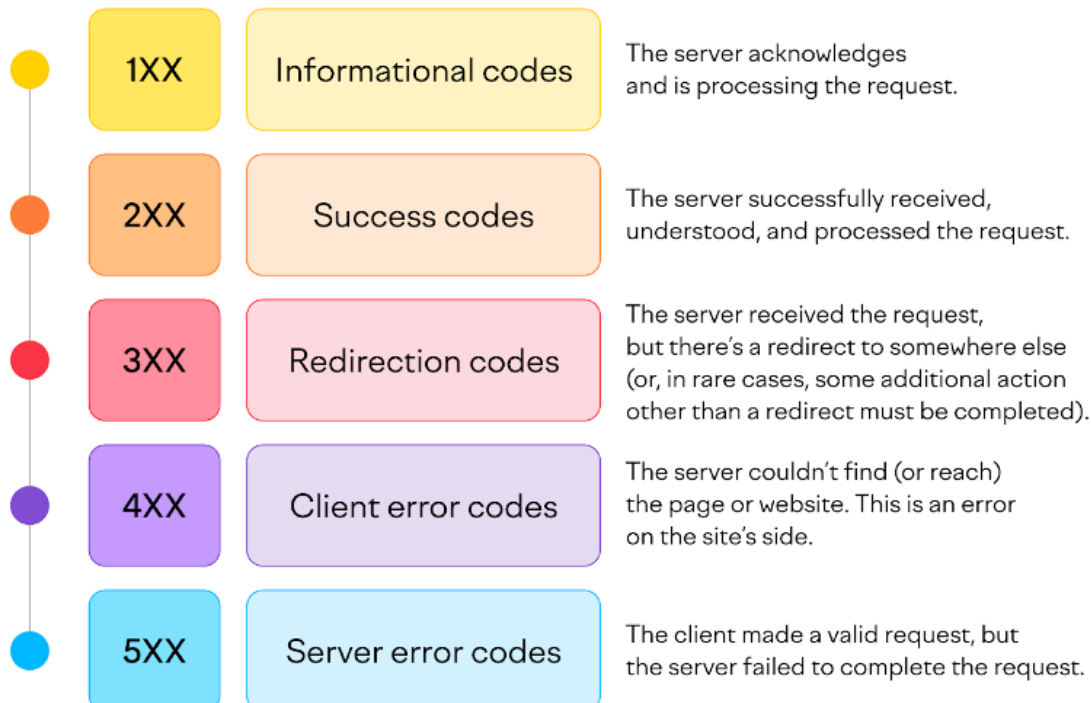
PATCH. Работает таким же образом, как и **PUT**, только по отношению к части ресурса.

DELETE. Клиент сообщает о том, что хотел бы удалить некий ресурс.

- **TRACE.** С помощью него можно проверить, изменяют ли промежуточные узлы в сети запрос клиента.
- **CONNECT.** Запускает туннель между клиентом и сервером. На картинке ниже изображена таблица, которая сравнивает основные **HTTP-методы**, такие как **GET, POST, PUT и PATCH**. Такая таблица нужна, чтобы правильно выбирать методы **HTTP** в зависимости от задачи.

HTTP Method	Request has body	Response has body	Safe	Idempotent	Cachable
GET	NO	YES	YES	YES	YES
POST	YES	YES	NO	NO	YES
PUT	YES	YES	NO	YES	NO
DELETE	YES	YES	NO	YES	NO

Коды состояния



100. Такой код характеризует то, что **HTTP-сообщение** клиента успешно принято и он может дальше отправлять их.

101. В **HTTP-запросе** можно попросить сервер переключиться на другую версию протокола, используя заголовок **Upgrade**. Если сервер удовлетворит такой запрос, то в ответе пришлёт код **101**.

102. Используется в работе **HTTP-запросов**, которые сервер принял, но ещё не успел обработать.

200 OK. Самый распространённый код состояния. Он подразумевает, что запрос клиента выполнен успешно.

201. Такой код используется в тех случаях, если с помощью запроса создаётся новый ресурс. Например, с помощью метода **PUT**. Если операция выполнена успешно, в ответе будет указан такой код.

301 Moved Permanently. Используется в тех случаях, когда запрашиваемый ресурс переместили в другую директорию навсегда — её можно будет найти в заголовке **Location**. Также у этого кода состояния есть похожий код **308**, который также означает, что клиент должен использовать такой же метод к новому местоположению ресурса.

302 Found. Этот код состояния применяется тогда, когда запрашиваемый ресурс в текущий момент временно размещён на другом адресе. Его укажут в заголовке **Location**. Ещё у этого кода состояния есть похожий код **307**, который используется в тех случаях, когда клиент должен использовать такой же метод.

400 Bad Request. Клиент составил своё **HTTP-сообщение** неправильно.

403 Access Forbidden. К ресурсу необходимы другие права доступа.

404 Not Found. Такой код состояния встречается чаще всего и означает, что сервер принял запрос, но по указанному адресу ничего не обнаружил.

500 Internal Server Error. Сервер не смог обработать запрос из-за внутренней ошибки. Например, в PHP-коде, который используется для обработки запроса, есть проблемы.

502 Bad Gateway и 504 Gateway Time Out. Иногда сервер выступает в роли промежуточного узла. Если следующий узел вернёт ошибку, сервер в ответе укажет код состояния **502**. Если не ответит за отведённое время, то **504**.

503 Service Unavailable. Такой код состояния означает, что сейчас на сервере технические неполадки и он не может обработать запрос.

Эндпоинты (конечные точки) — это URL-адреса, через которые клиенты взаимодействуют с сервером или **API**. Они определяют, какие данные или действия доступны и как к ним обратиться. Каждый эндпоинт соответствует определённому ресурсу (например, пользователю, товару) или операции (создание, удаление) и работает с конкретными **HTTP-методами (GET, POST, PUT, DELETE)**.

Как работают эндпоинты?

1. Маршрутизация: Сервер использует эндпоинты, чтобы определить, какой код выполнить при получении запроса. Например, запрос **GET /posts** может вызывать функцию, которая возвращает список постов из базы данных.

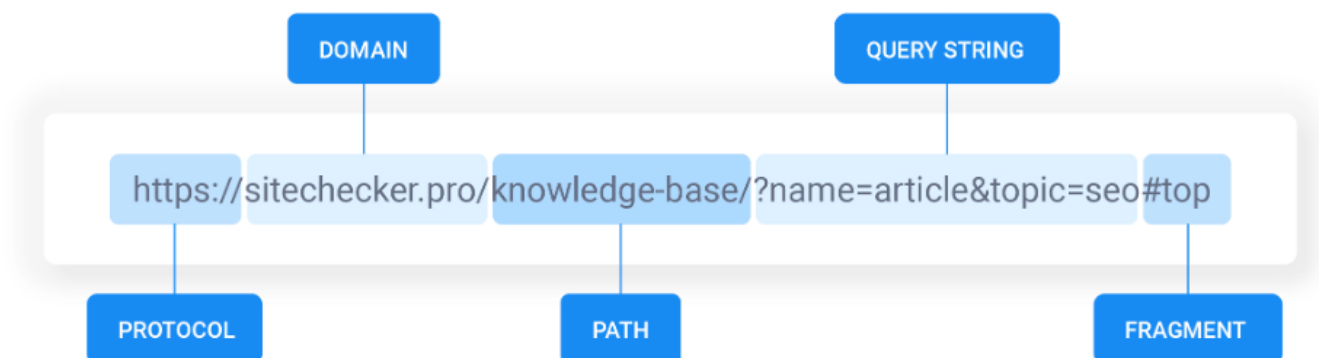
2. HTTP-методы: Каждый эндпоинт связан с определённым методом:

- **GET** — запрос данных.
- **POST** — отправка данных (создание).

- **PUT/PATCH** — обновление данных.
- **DELETE** — удаление.

3. Параметры: Эндпоинты могут включать параметры для уточнения запроса:

- **Path-параметры:** **GET** `/users/123` (ID пользователя — 123).
- **Query-параметры:** **GET** `/users?role=admin` (фильтрация по роли).



Зачем нужны эндпоинты?

- **Доступ к данным:** позволяют клиентам (приложениям, браузерам) получать или изменять информацию на сервере.
- **Структура API:** Четкие эндпоинты делают **API** понятным и предсказуемым (например, **RESTful API**).
- **Разделение ответственности:** Каждый эндпоинт отвечает за свою часть логики.

Что такое Swagger?

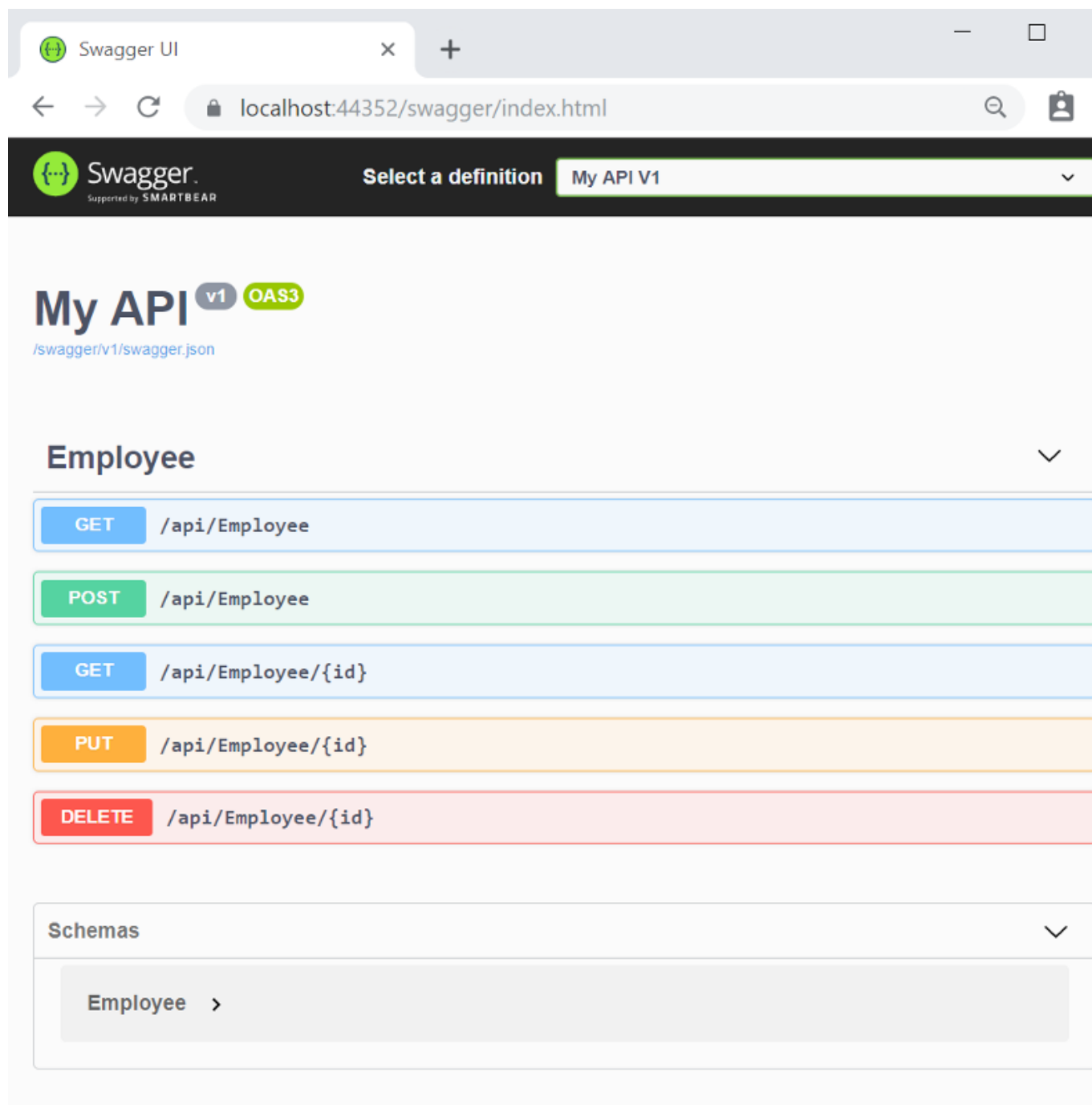
Swagger — это профессиональный набор инструментов для разработчиков **API**. Данный набор инструментов активно разрабатывается **SmartBear Software** и поддерживается сообществом открытого исходного кода (Open Source).

OpenAPI — это спецификация для описания **API**. На текущий момент времени актуальная версия **OpenAPI** — 3.1.0.

Swagger использует спецификацию **OpenAPI** для описания и документирования **API**, а инструменты **Swagger** позволяют использовать эту спецификацию для создания и тестирования **API**, а также для генерации клиентского кода.

Набор инструментов **Swagger** включает в себя следующие наиболее используемые инструменты:

1. **Swagger Editor** — редактор для разработки API-интерфейсов в соответствии со спецификацией **Open API**.
2. **Swagger UI** — веб-приложение, позволяющая визуализировать определения спецификаций **Open API** в интерактивном пользовательском интерфейсе.
3. **Swagger Codegen** — создание серверных заглушек и клиентских **SDK** пакетов на основе определений спецификаций **Open API**.



Основные компоненты документации

Общая информация:

- **Название API:** Укажите имя вашего **API**.
- **Версия API:** Например, 1.0.0.
- **Описание:** Кратко опишите назначение **API**.

- **Контакты:** Электронная почта или ссылка на поддержку.
- **Лицензия:** Если **API** распространяется под лицензией. Серверы
- **Базовый URL:** Адрес, по которому доступен **API** (например, <https://api.example.com/v1>).
- **Описание серверов:** Если **API** работает на нескольких окружениях (тестовое, продакшен).

Пути (Endpoints)

- **URL пути:** Например, `/users` или `/products/{id}`.
- **HTTP-методы:** GET, POST, PUT, DELETE и т.д.

Параметры:

- **Path-параметры:** Передаются в **URL** (например, `/users/{id}`).
- **Query-параметры:** Указываются в строке запроса (например, `?page=1`).
- **Body-параметры:** Данные, передаваемые в теле запроса (для **POST/PUT**).
- **Заголовки (Headers):** например, **Authorization** для аутентификации. Ответы
- **Коды состояния:** 200 (OK), 400 (Bad Request), 401 (Unauthorized) и т.д.
- **Формат ответа:** JSON, XML или другой.
- **Схема данных:** Описание структуры возвращаемых данных (например, список пользователей).

Модели данных (Schemas)

Определите структуры данных, используемые в запросах и ответах (например, `User`, `Product`).

Начните с базовой структуры:

```
openapi: 3.0.0
info:
  title: Example API
  version: 1.0.0
  description: API для управления пользователями
servers:
  - url: https://api.example.com/v1
paths: {}
components: {}
```

Добавьте пути (Endpoints). **Пример для GET-запроса:**

```
paths:
  /users:
    get:
      summary: Получить список пользователей
      parameters:
        - name: limit
          in: query
          description: Лимит результатов
          schema:
            type: integer
```

```
responses:
  '200':
    description: Успешный ответ
    content:
      application/json:
        schema:
          type: array
          items:
            $ref: '#/components/schemas/User'
```

Опишите модели данных:

```
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: integer
        name:
          type: string
        email:
          type: string
```

Лучшие практики

- **Четкость:** Каждый **endpoint** должен иметь описание, включая примеры использования.
- **Детализация параметров:** Укажите тип данных, обязательность, допустимые значения.
- **Примеры ответов:** Добавьте примеры успешных и ошибочных ответов.
- **Валидация:** Проверьте документацию через валидаторы (например, **Swagger Validator**).
- **Версионирование:** Храните документацию в системе контроля версий (**Git**).

Интеграция и тестирование

- **Swagger UI:** Загрузите вашу спецификацию в Swagger UI, чтобы получить интерактивную документацию.
- **Тестирование:** Используйте встроенные инструменты Swagger для отправки тестовых запросов.
- **Обновление:** Регулярно синхронизируйте документацию с изменениями в **API**.

Определение конечной точки

Конечная точка **API** — это **URL-адрес**, по которому сервер или сервис предоставляет доступ к данным или функционалу. В разработке **API** такие конечные точки часто называют «ручками». То есть, **API** можно «дергать» разными способами для получения различных данных или выполнения действий.

В **FastAPI** конечные точки определяются с помощью Python-функций, которые привязываются к HTTP-методам, таким как `@app.get()` для обработки запросов **GET** или `@app.post()` для обработки запросов **POST** и так далее. Эти функции обрабатывают входящие запросы и генерируют ответы.

Для начала рассмотрим два простых примера: один для получения данных с сервера с помощью **GET-запроса**, а другой — для отправки данных с помощью **POST-запроса**.

Пример:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

fake_db = [{"username": "vasya", "user_info": "любит колбасу"},
{"username": "katya", "user_info": "любит петь"}]

# Обрабатываем GET-запрос, чтобы вернуть список пользователей
@app.get('/users')
async def get_all_users():
    return fake_db

# Обрабатываем POST-запрос, чтобы добавить нового пользователя
@app.post('/add_user')
async def add_user(username: str, user_info: str):
    fake_db.append({"username": username, "user_info": user_info})
    return {"message": "Юзер успешно добавлен в базу данных"}
```

В этом примере:

- Мы создаём **GET-метод** для получения списка пользователей с сервера.
- Создаём **POST-метод** для добавления нового пользователя в «фейковую» базу данных. Мы передаем параметры как query-параметры (параметры, передаваемые в URL), но для **POST-запросов** это не самый привычный способ. Обычно в **POST-запросах** данные передаются в теле запроса (чаще всего в формате JSON).

Важно: Не пугайтесь, если пока не всё понятно. Мы расскажем и покажем, как правильно отправлять и обрабатывать данные в теле запроса.

Параметры запроса, пути и тела в FastAPI

В **FastAPI** вы можете работать с разными типами параметров, которые могут быть частью запроса. Существует три основных типа параметров: **параметры пути**, **параметры запроса** и **параметры тела запроса**. Все они выполняют разные функции и используются в разных ситуациях.

Параметры пути (Path Parameters)

Параметры пути — это части URL, которые захватываются и передаются в виде аргументов функции. Эти параметры часто используются для уникальной идентификации ресурса, например, для получения информации о конкретном пользователе по его **ID**.

Пример: `/users/{user_id}`, где `{user_id}` — это параметр пути.

Параметры запроса (Query Parameters)

Параметры запроса — это дополнительные данные, которые передаются в **URL** в формате ключ-значение после знака вопроса `?`. Эти параметры часто используются для фильтрации, сортировки или ограничения количества данных, которые возвращает сервер.

Пример: `/users?limit=10&active=true`, где `limit` и `active` — параметры запроса.

Параметры тела запроса (Body Parameters)

Параметры тела запроса передаются в теле HTTP-запроса, обычно в формате **JSON**, и используются для отправки больших данных, таких как объекты или списки. Эти параметры часто используются в **POST**, **PUT** и **PATCH** запросах, когда нужно отправить более сложные данные для создания или обновления ресурсов.

Пример: тело запроса может содержать информацию о пользователе, такую как имя и адрес электронной почты, в формате **JSON**.

В следующих практиках мы подробно рассмотрим, как работать с каждым из этих типов параметров в **FastAPI**, как их обрабатывать и как использовать для создания гибких и функциональных **API**.

Параметры пути в FastAPI

В **FastAPI** параметры пути — это части **URL**, которые позволяют передавать динамические данные в запросе. Они используются для захвата информации из **URL**, такой как идентификаторы объектов или другие переменные значения.

Параметры пути заключаются в фигурные скобки `{}`, и они являются обязательной частью пути **URL**. При обращении к конечной точке, значение этих параметров передается в виде аргументов функции.

Пример использования параметра пути

Допустим, у нас есть **API**, который позволяет получить информацию о пользователях по их идентификатору. Путь будет выглядеть так:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get('/users/{user_id}') # тут объявили параметр пути
async def get_user(user_id: int): # тут указали его тип данных
    # какая-то логика работы поиска
```

```
return {"user_id": user_id, "message": f"Информация о пользователе с ID {user_id}"}
```

Здесь `{user_id}` — это параметр пути, который будет передан в функцию `get_user` как аргумент `user_id`. В зависимости от того, какой идентификатор вы укажете в **URL**, будет возвращаться соответствующая информация.

Как работает запрос

Предположим, что ваше приложение запущено на `http://localhost:8000`, и вы хотите получить информацию о пользователе с **ID 5**. Вы отправляете **GET-запрос** по следующему **URL**:

```
http://localhost:8000/users/5
```

Ответ будет:

```
{
  "user_id": 5,
  "message": "Информация о пользователе с ID 5"
}
```

Если вы попытаетесь использовать другой идентификатор, например, 10, результат изменится:

```
http://localhost:8000/users/10
```

Ответ:

```
{
  "user_id": 10,
  "message": "Информация о пользователе с ID 10"
}
```

Как указывать тип данных для параметров пути

FastAPI автоматически проверяет типы данных параметров пути. В примере выше мы указали, что параметр `user_id` должен быть целым числом (`int`). Если вы передадите неправильный тип данных, например строку вместо числа, **FastAPI** автоматически вернет ошибку.

Пример:

```
http://localhost:8000/users/abc
```

Ответ:

```
{
  "detail": [
    {
      "loc": ["path", "user_id"],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ]
}
```

FastAPI предоставляет множество полезных функций для работы с параметрами пути, позволяя вам быстро создавать динамические и удобные **API**.

Применение параметров пути в реальных приложениях

Параметры пути особенно полезны, когда вы хотите идентифицировать конкретный ресурс или объект, например:

- Получение информации о пользователе по его **ID**.
- Получение информации о товаре по его **SKU**.
- Удаление объекта из базы данных по уникальному идентификатору.

Пример с удалением пользователя:

```
# продолжение main.py
@app.delete('/users/{user_id}')
async def delete_user(user_id: int):
    return {"message": f"Пользователь с ID {user_id} был удален"}
```

Таким образом, параметры пути — это удобный способ динамично работать с **URL** и извлекать данные из пути запроса, обеспечивая гибкость в создании **API**.

Создание новой конечной точки для работы с фейковой базой данных

В этом шаге мы создадим конечную точку, которая будет обрабатывать запросы **GET** для получения информации о пользователе. Мы будем работать с фейковой базой данных (условным словарем), чтобы показать, как можно извлекать данные по идентификатору пользователя.

Обновим файл **main.py**:

```
# main.py
from fastapi import FastAPI

app = FastAPI()
```

```
# Пример фейковой базы данных
fake_users = {
    1: {"username": "john_doe", "email": "john@example.com"},
    2: {"username": "jane_smith", "email": "jane@example.com"},
}

# Конечная точка для получения информации о пользователе по ID
@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id in fake_users:
        return fake_users[user_id]
    return {"error": "User not found"}
```

В этой конечной точке мы используем параметр пути `user_id`, чтобы искать пользователя в словаре `fake_users`. Если пользователь найден, мы возвращаем его данные. Если нет — возвращаем условную ошибку с сообщением `"User not found"`. Как работать с реальными ошибками будет показано позже.

Теперь, при обращении по **URL** вида `http://localhost:8000/users/1`, вы получите информацию о первом пользователе, а если указать несуществующий **ID**, например `http://localhost:8000/users/99`, будет возвращена ошибка.

Этот пример демонстрирует, как можно взаимодействовать с базой данных или фейковыми данными в **FastAPI**, выполняя операции поиска или извлечения информации.

Параметры запроса в FastAPI

В **FastAPI** параметры запроса используются для настройки поведения конечной точки **API** в зависимости от значений, передаваемых в **URL**. Они позволяют управлять тем, как сервер обрабатывает запросы, без необходимости изменения самого пути. Параметры запроса передаются в **URL** после знака вопроса `?`, и они состоят из пар "ключ-значение", например: `limit=10`.

Например, если вы хотите ограничить количество возвращаемых пользователей, вы можете использовать параметр запроса `limit`. В этом случае **URL** будет выглядеть так: `http://localhost:8000/users/?limit=3`.

Пример работы с параметром запроса

Предположим, что у нас есть приложение, которое возвращает список пользователей. Вначале мы добавим параметр запроса, чтобы можно было ограничивать количество возвращаемых пользователей. Вот как это будет выглядеть:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

# Пример пользовательских данных (для демонстрационных целей)
fake_users = {
    1: {"username": "john_doe", "email": "john@example.com"},
    2: {"username": "jane_smith", "email": "jane@example.com"},
    3: {"username": "john_doe", "email": "john@example.com"},
    4: {"username": "jane_smith", "email": "jane@example.com"},
    5: {"username": "john_doe", "email": "john@example.com"},
    6: {"username": "jane_smith", "email": "jane@example.com"},
    7: {"username": "john_doe", "email": "john@example.com"},
    8: {"username": "jane_smith", "email": "jane@example.com"},
    9: {"username": "john_doe", "email": "john@example.com"},
    10: {"username": "jane_smith", "email": "jane@example.com"},
}
```

```
2: {"username": "jane_smith", "email": "jane@example.com"},
3: {"username": "alice_jones", "email": "alice@example.com"},
4: {"username": "bob_white", "email": "bob@example.com"},
}

# Конечная точка для получения списка пользователей с параметром запроса
limit
@app.get("/users/")
def read_users(limit: int = 10):
    # Ограничиваем количество пользователей, используя параметр limit
    return dict(list(fake_users.items())[:limit])
```

Как работает параметр запроса

В этом примере мы добавили параметр запроса `limit`, который ограничивает количество пользователей, которые будут возвращены в ответе. Если вы отправите запрос:

```
http://localhost:8000/users/?limit=3
```

Ответ будет содержать только первых трех пользователей:

```
{
  "1": {"username": "john_doe", "email": "john@example.com"},
  "2": {"username": "jane_smith", "email": "jane@example.com"},
  "3": {"username": "alice_jones", "email": "alice@example.com"}
}
```

Если вы не передадите параметр `limit`, то по умолчанию вернется 10 пользователей (как указано в аргументе функции):

```
http://localhost:8000/users/
```

Ответ:

```
{
  "1": {"username": "john_doe", "email": "john@example.com"},
  "2": {"username": "jane_smith", "email": "jane@example.com"},
  "3": {"username": "alice_jones", "email": "alice@example.com"},
  "4": {"username": "bob_white", "email": "bob@example.com"}
}
```

Поясним по махинациям

Мы из условного словаря `fake_users` сделали массив, обрезали его через слайсы и потом обернули обратно в словарь, чтобы вернуть ответ по формату **JSON**, который сейчас является негласным стандартом обмена данными через **API**. В реальных приложениях данные с высокой долей вероятности будут подтягиваться из базы данных и форматироваться согласно объявленным моделям.

Как использовать параметры запроса для фильтрации данных

Параметры запроса также полезны для фильтрации данных, сортировки или добавления дополнительной логики обработки. Например, вы можете добавить несколько параметров, чтобы фильтровать список пользователей по различным признакам, таким как `username` или `email`:

```
# обновление маршрута main.py
@app.get("/users/")
def read_users(username: str = None, email: str = None, limit: int = 10):
    filtered_users = fake_users

    if username:
        filtered_users = {key: user for key, user in
            filtered_users.items() if username.lower() in user["username"].lower()}

    if email:
        filtered_users = {key: user for key, user in
            filtered_users.items() if email.lower() in user["email"].lower()}

    return dict(list(filtered_users.items())[:limit])
```

Теперь вы можете фильтровать пользователей по имени или `email`, передавая параметры запроса в **URL**. Например, чтобы получить пользователей с именем, содержащим `"john"`:

```
http://localhost:8000/users/?username=john
```

Ответ:

```
{
  "1": {"username": "john_doe", "email": "john@example.com"}
}
```

Параметры запроса в **FastAPI** — это мощный инструмент для динамической настройки поведения конечных точек **API**. Вы можете использовать их для фильтрации, ограничения или изменения обработки данных, не изменяя структуру **URL**. Эти параметры легко использовать и они делают ваш API более гибким и удобным для пользователей.

Работа с телом запроса и ответов в FastAPI

Тело запроса — это часть **HTTP-запроса**, где передаются данные от клиента (например, браузера или другого сервера) на сервер. Тело запроса используется в основном в запросах типов **POST**, **PUT**, **PATCH**, а также некоторых других, где необходимо передать данные, которые не могут быть выражены непосредственно в **URL** (например, через параметры пути или запроса). Это может быть информация о новом пользователе, данные для обновления записи в базе данных или другие данные.

Примеры запросов, которые обычно используют тело запроса:

- **POST** — создание нового ресурса, например, создание нового пользователя или поста.
- **PUT** — обновление существующего ресурса.
- **PATCH** — частичное обновление ресурса.

Когда мы отправляем запрос с телом, оно может быть в разных форматах, но наиболее популярным является **JSON** (JavaScript Object Notation). **JSON** стал стандартом обмена данными между клиентом и сервером благодаря своей легкости и удобочитаемости.

Почему JSON?

JSON — это текстовый формат, который легко читать и писать, а также легко парсится во всех популярных языках программирования. В **API JSON** используется для передачи данных, поскольку он является стандартом в веб-разработке и предоставляет удобную структуру для сериализации и десериализации данных.

Когда вы отправляете **POST-запрос** с данными, вы, как правило, используете **JSON** для того, чтобы данные могли быть интерпретированы сервером. Например, тело запроса может содержать информацию о пользователе в следующем формате:

```
{
  "username": "john_doe",
  "user_info": "любит петь"
}
```

Отправка тела запроса

Тело запроса отправляется в **HTTP-запросе**. Когда вы используете инструменты, такие как **Postman** или **curl**, вы можете явно указать тело запроса, например, отправив **JSON-данные** в теле **POST-запроса**. В веб-приложениях, если вы используете **JavaScript**, это также можно сделать через **fetch** или **XMLHttpRequest**.

Пример отправки запроса через **curl**:

```
curl -X POST "http://localhost:8000/add_user" -H "Content-Type: application/json" -d '{"username": "john_doe", "user_info": "любит петь"}'
```

Пример отправки запроса с использованием **JavaScript (fetch)**:


```
fetch('http://localhost:8000/add_user', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        username: 'john_doe',
        user_info: 'любит петь'
    })
});
```

Работа с телом запроса в FastAPI

В **FastAPI** для работы с телом запроса вы используете **модели Pydantic**. Эти модели описывают структуру данных, которые вы ожидаете получить, и позволяют **FastAPI** автоматически проверять их на соответствие. Это также позволяет **FastAPI** автоматически генерировать документацию, в которой будет описано, какие поля ожидаются.

В примере ниже создается модель **User**, которая будет описывать данные, которые ожидаются в теле запроса.

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Создание модели данных для пользователя
class User(BaseModel):
    username: str
    user_info: str

fake_db = [{"username": "vasya", "user_info": "любит колбасу"},
{"username": "katya", "user_info": "любит петь"}]

@app.get('/users')
async def get_all_users():
    return fake_db

# Конечная точка для добавления нового пользователя
@app.post('/add_user')
async def add_user(user: User): # Используем модель для валидации данных
    fake_db.append({"username": user.username, "user_info":
user.user_info})
    return {"message": "Юзер успешно добавлен в базу данных"}
```

Как работает валидация входящих данных?

При отправке **POST-запроса** с телом, которое соответствует модели **User**, **FastAPI** автоматически валидирует данные. Если в теле запроса будут переданы некорректные данные (например, если тип данных не соответствует ожидаемому), **FastAPI** автоматически вернёт ошибку. Это помогает избежать ошибок в процессе работы с данными, а также значительно упростить обработку запросов.

Пример корректного тела запроса:

```
{
  "username": "john_doe",
  "user_info": "любит петь"
}
```

Если же тип данных или структура будут некорректными, например, если вы отправите число вместо строки, **FastAPI** вернёт ошибку с описанием проблемы.

Ответы с использованием моделей Pydantic

Так же, как мы используем модели для валидации входящих данных, мы можем использовать их для формата ответов. Для этого используется параметр **response_model** в декораторе конечной точки.

```
# обновляем маршрут в main.py
@app.post('/add_user', response_model=User) # Указываем модель для ответа
async def add_user(user: User):
    fake_db.append({"username": user.username, "user_info":
user.user_info})
    return user # Возвращаем добавленного пользователя в формате,
указанном в response_model
```

Теперь при успешном добавлении пользователя в фейковую базу данных, **FastAPI** автоматически вернёт данные, соответствующие модели **User**. Ответ будет иметь следующий вид:

```
{
  "username": "john_doe",
  "user_info": "любит петь"
}
```

Это позволяет обеспечить согласованность данных как на входе, так и на выходе, делая ваше **API** более предсказуемым и безопасным.

Работа с телом запроса и ответами является важным аспектом в разработке **API**. Используя **модели Pydantic**, вы получаете мощный инструмент для валидации и форматирования данных, что существенно улучшает качество и поддержку вашего кода.

Тестирование конечных точек в FastAPI

Тестирование — важная часть разработки **API**. В **FastAPI** можно тестировать конечные точки с помощью встроенной документации, сторонних инструментов и интеграционных тестов в коде.

Перед началом тестирования полезно ознакомиться с примером приложения, включающего работу с параметрами пути, запроса и тела:

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Модель для валидации тела запроса
class User(BaseModel):
    username: str
    user_info: str

# Фейковая база данных
fake_db = [
    {"username": "vasya", "user_info": "любит колбасу"},
    {"username": "katya", "user_info": "любит петь"}
]

# Получение пользователя по параметру пути
@app.get('/users/{username}')
async def get_user(username: str):
    for user in fake_db:
        if user["username"] == username:
            return user
    return {"error": "User not found"}

# Получение списка пользователей с ограничением (параметр запроса)
@app.get("/users/")
async def read_users(limit: int = 10):
    return fake_db[:limit]

# Добавление нового пользователя (параметр тела запроса)
@app.post('/add_user', response_model=User)
async def add_user(user: User):
    fake_db.append({"username": user.username, "user_info":
user.user_info})
    return user
```

Запуск приложения

Для запуска сервера **FastAPI** выполните в терминале команду:

```
uvicorn main:app --reload
```

После запуска сервер будет доступен по адресу `http://localhost:8000`.

Использование инструментов для тестирования

FastAPI автоматически генерирует документацию **API**, что позволяет легко тестировать конечные точки.

Swagger UI и ReDoc

- **Swagger UI:** Доступен по адресу `http://localhost:8000/docs`. Здесь можно просматривать описание всех конечных точек, заполнять поля запроса и отправлять их для получения ответа в формате **JSON**.
- **ReDoc:** Доступен по адресу `http://localhost:8000/redoc`. Предоставляет компактное и строгие представление документации **API**.

Сторонние инструменты

Помимо встроенной документации, тестировать API можно с помощью инструментов:

- **Postman**
 1. Откройте Postman.
 2. Создайте новый запрос, указав тип (**GET**, **POST** и т.д.) и URL (например, `http://localhost:8000/users/artur4ik` для получения информации о пользователе).
 3. При необходимости добавьте параметры, заголовки или тело запроса.
 4. Нажмите **"Send"** для отправки запроса и просмотрите ответ.

- **curl**

Для работы в терминале можно использовать **curl**:

- **GET-запрос** для получения пользователя:

```
curl "http://localhost:8000/users/artur4ik"
```

- **GET-запрос** с параметром запроса:

```
curl "http://localhost:8000/users/?limit=2"
```

- **POST-запрос** для добавления пользователя:

```
curl -X POST "http://localhost:8000/add_user" \  
-H "Content-Type: application/json" \  
-d '{"username": "rustam", "user_info": "любитель buble tea"}'
```

Для интеграционного тестирования **FastAPI** рекомендуется использовать [pytest](#) совместно с [TestClient](#) из **FastAPI**.

Made with ❤️ by **dvOretsky**